

# Trabajo Práctico 7 — Sobreviví a los slimes (un *Vampire Survivors* básico)

**Diplomatura de Videojuegos · Clase 7** Objetivo: armar un juego de **supervivencia**: un caballero en una arena, **hordas de slimes** que aparecen por los bordes y lo persiguen, y un arma que **dispara sola** al enemigo más cercano. Cada tanto aparece un **slime elite**, más duro, con su **barra de vida sobre la cabeza**. Es el TP que junta **todo** lo que vimos: movimiento, señales, herencia, instanciación, spawning y HUD.

## Qué vas a lograr

- Un **jugador** que se mueve en 4 direcciones, animado, encerrado en la arena.
- **Slimes** que aparecen por los bordes de la pantalla y **te persiguen** (la IA más simple que existe).
- **Balas automáticas**: cada medio segundo sale una hacia el enemigo **más cercano**, sin apretar nada.
- Un **slime elite** que aparece cada 8 enemigos: más lento, aguanta 5 balas, y muestra una **barra de vida** encima.
- Un **HUD** con tu vida y la cuenta de slimes eliminados. Si tu vida llega a 0, la partida se reinicia.

 **¿Qué es un *Vampire Survivors*?** Un género de 2022 que explotó por lo simple: el personaje **ataca solo**, vos solo te movés, y la única meta es **sobrevivir** al enjambre el mayor tiempo posible. *Brotato* (hecho en Godot) y *Halls of Torment* son del mismo palo.

 **Tiempo estimado:** 90–120 min. Es el TP más largo, pero cada parte se prueba sola. **Escribí el código vos.**

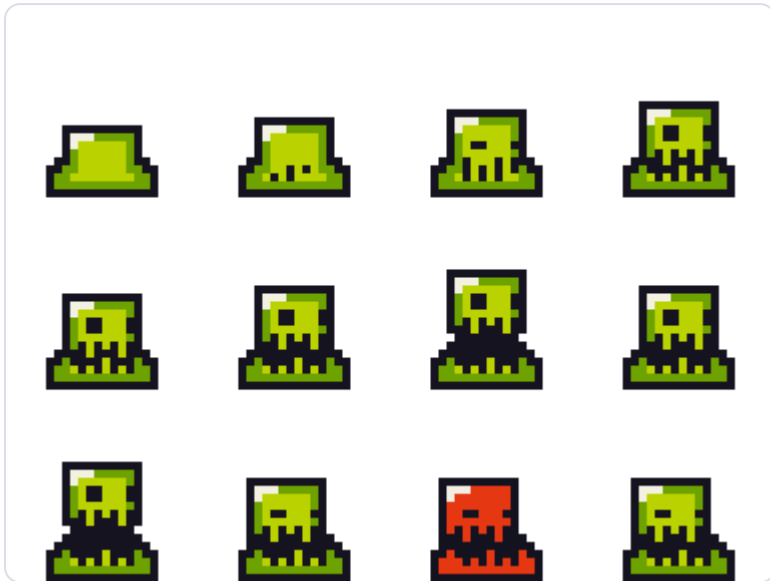
 **Viene de todo el curso:** movimiento (TP3), [Area2D](#) y señales (TP4), **herencia**, [preload](#) + [instantiate](#) y **spawning** con [Timer](#) (TP5), y [ProgressBar](#) + HUD (Clase 6).

## Los assets (Brackeys · CC0)

Están en [tp7-assets/](#). Licencia **CC0** (uso libre, [créditos acá](#)).

- [knight.png](#) — el jugador, el mismo caballero del TP4 (hoja 8×8, frames de 32×32, con texto incrustado que ignorás).
- [slime\\_green.png](#) — el enemigo básico. Hoja de 4×3, frames de 24×24, sin texto. La **fila del medio** es la animación de caminar.

- `slime_purple.png` — el enemigo élite. **Misma hoja**, otro color.
- `bala.png` — un puntito de 8×8. Se escala en el editor.



---

## ✦ Cómo va a quedar el proyecto

---

Cuatro escenas: el nivel, y tres “moldes” que se instancian en vivo.

```

nivel.tscn
  Nivel (Node2D)
  |— Jugador (CharacterBody2D) [grupo "jugador"]
  |   |— AnimatedSprite2D ← caballero idle / run
  |   |— CollisionShape2D
  |   |— TimerDisparo (Timer) ← dispara sola cada 0.4 s
  |— Spawner (Node2D)
  |   |— Timer ← saca un slime por segundo
  |— Enemigos (Node2D) ← contenedor: acá van los slimes
  |— Balas (Node2D) ← contenedor: acá van las balas
  |— HUD (CanvasLayer)
  |   |— BarraVida (ProgressBar)
  |   |— LabelKills (Label)

enemigo.tscn      Enemigo (Area2D) [grupo "enemigo"] + AnimatedSprite2D + CollisionShape2D
enemigo_elite.tscn igual, con slime violeta + BarraVida (ProgressBar) ← HEREDA de enemigo.gd
bala.tscn         Bala (Area2D) + Sprite2D + CollisionShape2D + TimerVida (Timer)

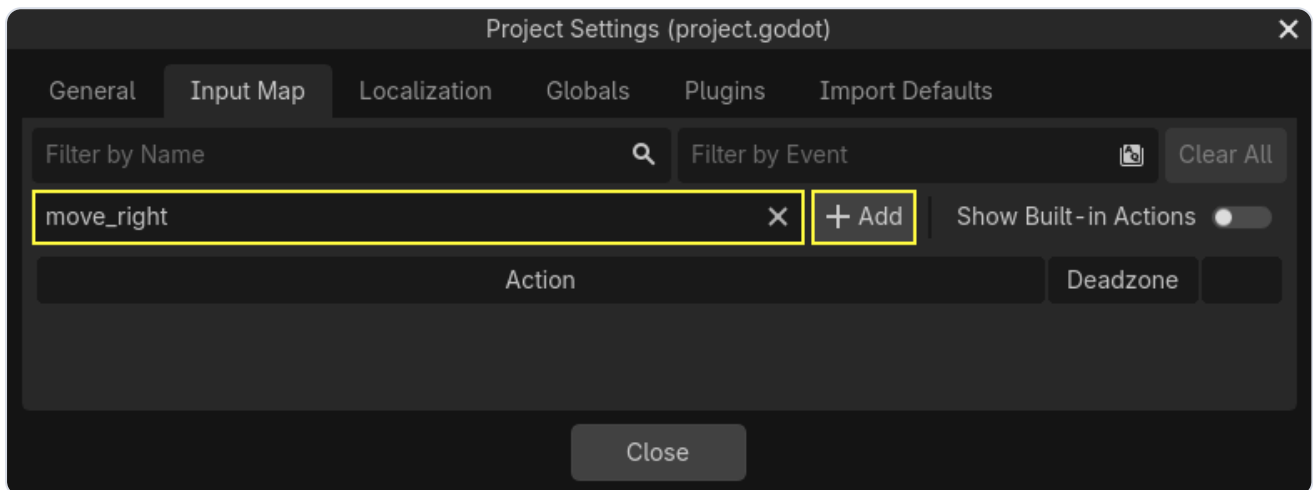
```

🗨️ **Quién habla con quién.** El **slime** busca al jugador por su **grupo** y lo persigue; al tocarlo le hace daño (señal `body_entered`). La **bala** vuela recta y, al tocar un slime, le resta vida (señal `area_entered`). Nadie tiene una referencia “dura” a nadie: todo es **grupos + señales**, como en la Clase 4.

## 🔧 Parte 0 — Proyecto, assets y controles

1. Godot → **New Project** → `tp7-sobrevivir` → carpeta vacía → **Create & Edit**.
2. Arrastrá al **FileSystem** los cuatro archivos de `tp7-assets/`: `knight.png`, `slime_green.png`, `slime_purple.png`, `bala.png`.
3. **Píxeles nítidos:** **Project** → **Project Settings** → **Rendering** → **Textures** → **Default Texture Filter** = **Nearest**.
4. **Input Map** (**Project** → **Project Settings** → **Input Map**): creá las cuatro acciones de siempre, las mismas del TP3:

| ACCIÓN          | TECLA |
|-----------------|-------|
| mover_derecha   | D o → |
| mover_izquierda | A o ← |
| mover_arriba    | W o ↑ |
| mover_abajo     | S o ↓ |

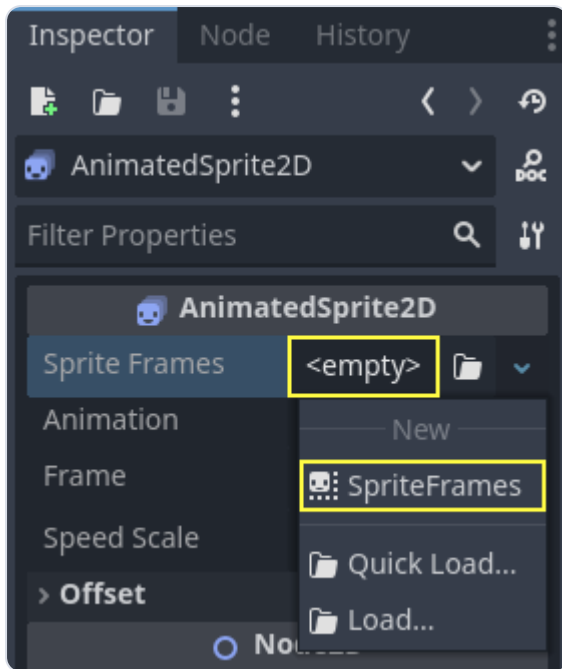


5. Creá la escena principal: **Otro Nodo** → `Node2D` → renombralo `Nivel` → guardá como `nivel.tscn`.
  6. Hijos de `Nivel`, dos `Node2D` vacíos: `Enemigos` y `Balas`. Son contenedores: ahí van a caer las instancias, para tener el árbol ordenado (el consejo de la Clase 5).
- ✓ **Punto de control 0:** tenés `nivel.tscn` con `Enemigos` y `Balas` vacíos, los 4 sprites en el FileSystem y las 4 acciones en el Input Map.

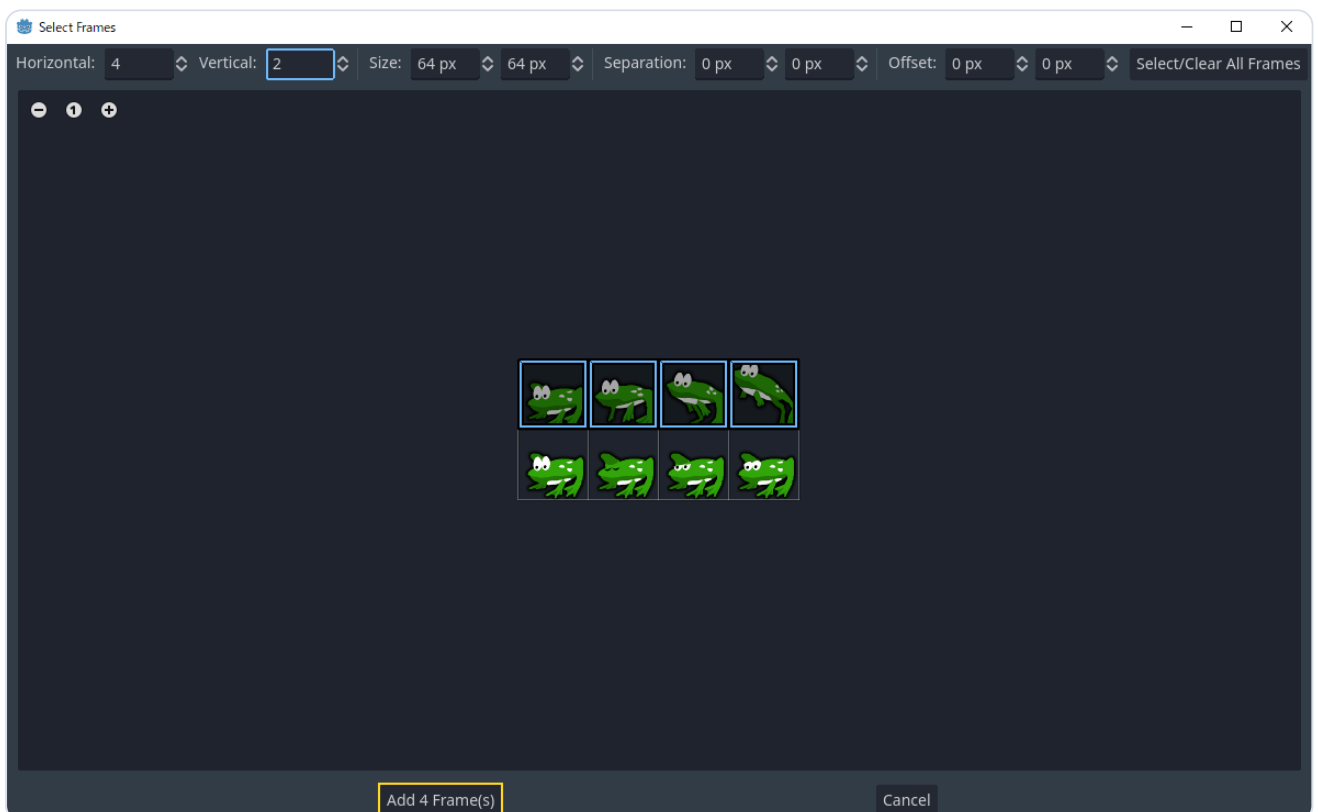
## Parte 1 — El jugador y su HUD

### 1.1 · El caballero

1. Hijo de `Nivel` → `CharacterBody2D` → renombralo `Jugador`. Ubicalo en el **centro** de la pantalla (Position ≈ `576, 324`).
2. Hijo de `Jugador` → `AnimatedSprite2D` → **Sprite Frames** → **Nuevo SpriteFrames**. Armá las dos animaciones **exactamente como en el TP4** (Parte 2):

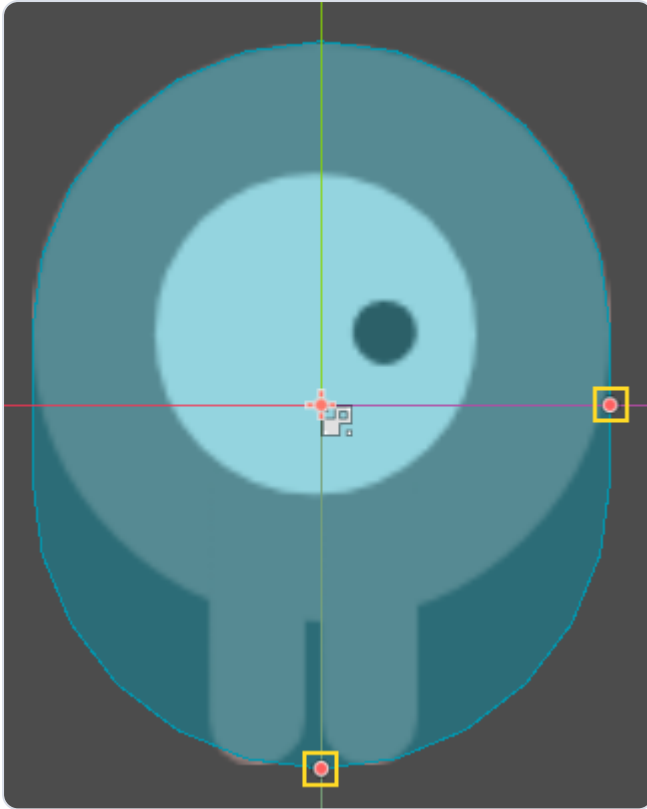


- **idle** : Añadir Frames desde un Sprite Sheet → **knight.png** , **8 × 8** → los 4 primeros frames de la fila de arriba.
- **run** : animación nueva → misma hoja → los 8 frames de la fila del **RUN**. FPS ≈ **10**.
- En las dos: **Loop**. En **idle** : **Autoplay on Load**.



Poné el **Scale** del **AnimatedSprite2D** en **2, 2** (el caballero es chiquito).

3. Hijo de **Jugador** → **CollisionShape2D** → **CapsuleShape2D** que envuelva al caballero.



## 1.2 · El HUD

4. Hijo de `Nivel` → `CanvasLayer` → renómbalo `HUD`.
5. Hijo de `HUD` → `ProgressBar` → renómbalo `BarraVida`. En el Inspector: **Min Value** `0`, **Max Value** `100`, **Value** `100`, **Show Percentage** apagado. Dale un tamaño de unos `220 x 22` y ponela arriba a la izquierda.
6. Hijo de `HUD` → `Label` → renómbalo `LabelKills`. **Text** = `Slimes: 0`. Debajo de la barra.

## 1.3 · El script del jugador

7. Clic derecho en `Jugador` → **Attach Script** → `res://jugador.gd`:

```

extends CharacterBody2D

var velocidad := 200.0
var vida := 100
var kills := 0

func _ready() -> void:
    add_to_group("jugador")
    actualizar_hud()

func _physics_process(delta: float) -> void:
    # 1) Dirección según las teclas (igual que en el TP3)
    var direccion := Vector2.ZERO
    if Input.is_action_pressed("mover_derecha"):
        direccion.x += 1
    if Input.is_action_pressed("mover_izquierda"):
        direccion.x -= 1
    if Input.is_action_pressed("mover_abajo"):
        direccion.y += 1
    if Input.is_action_pressed("mover_arriba"):
        direccion.y -= 1

    # 2) Mover con el motor de física (como en el TP4)
    velocity = direccion.normalized() * velocidad
    move_and_slide()

    # 3) Animación
    if direccion != Vector2.ZERO:
        $AnimatedSprite2D.play("run")
        $AnimatedSprite2D.flip_h = direccion.x < 0
    else:
        $AnimatedSprite2D.play("idle")

    # 4) No salir de la arena
    var limites := get_viewport_rect().size
    position.x = clamp(position.x, 0, limites.x)
    position.y = clamp(position.y, 0, limites.y)

func recibir_dano(cantidad: int) -> void:
    vida -= cantidad
    actualizar_hud()
    if vida <= 0:
        get_tree().reload_current_scene()    # se acabó: de vuelta a empezar

```

```
func sumar_kill() -> void:
    kills += 1
    actualizar_hud()

func actualizar_hud() -> void:
    get_node("../HUD/LabelKills").text = "Slimes: " + str(kills)
    get_node("../HUD/BarraVida").value = vida
```

### 🗨 Dos cosas para mirar:

- `.normalized()` deja el vector de dirección con largo 1. Sin eso, en diagonal irías más rápido (lo vimos en el Extra del TP3).
- `velocity` + `move_and_slide()` sin multiplicar por `delta`: el motor de física lo aplica por dentro. La regla de la Clase 3 ("siempre `* delta`") es para cuando movés `position` a mano — y eso sí lo vamos a hacer con los slimes y las balas.

✅ **Punto de control 1:** con **F6** el caballero camina en 4 direcciones, se anima, no sale de la pantalla, y el HUD dice `Slimes: 0` con la barra llena.

### 🚫 Se ve borroso / no se mueve / la barra no aparece

#### ▼ Abrí para ver soluciones

- Borroso → falta **Default Texture Filter = Nearest** (Parte 0).
- No se mueve → las acciones del Input Map tienen que llamarse **exactamente** `mover_derecha`, etc. Y hacé **click en la ventana del juego** para que reciba las teclas.
- Error `Node not found: ../HUD/LabelKills` → `HUD` tiene que ser **hermano** de `Jugador` (los dos hijos de `Nivel`), y los nombres, exactos.
- La barra se ve vacía → **Value** = 100 y **Max Value** = 100 en el Inspector.

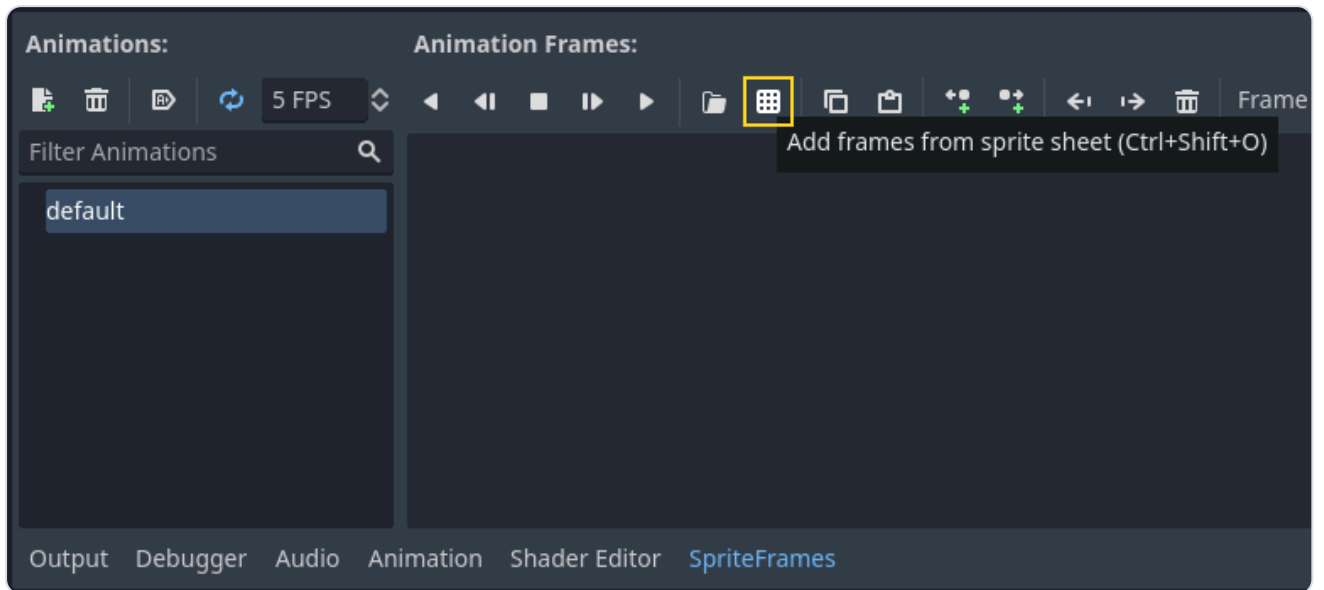
## 🟢 Parte 2 — El slime: la IA más simple que existe

**Concepto:** el enemigo es un `Area2D` que se mueve solo (como los objetos que caían en el TP5), pero en vez de caer, **persigue**. Toda su "inteligencia" es una línea: *calculá la dirección al jugador y avanzá*.

### 2.1 · La escena del slime

1. **Escena** → **Otro Nodo** → `Area2D` → renombralo `Enemigo`. Guardá como `enemigo.tscn` (escena aparte: es un **molde**).

2. Hijo → **AnimatedSprite2D** → **Nuevo SpriteFrames**. Renombrá la animación **default** a **caminar**.  
Añadir Frames desde un Sprite Sheet → **slime\_green.png**, **Horizontal = 4**, **Vertical = 3** → seleccioná los **4 frames de la fila del medio**. **Loop ON**, **FPS ≈ 8**. **Scale 2, 2**.



3. Hijo → **CollisionShape2D** → **CircleShape2D**, radio ≈ **20**, que cubra al slime.

## 2.2 · El script del slime

4. Clic derecho en **Enemigo** → **Attach Script** → **res://enemigo.gd**:

```

extends Area2D

var velocidad := 60.0
var vida := 1
var dano := 10

func _ready() -> void:
    add_to_group("enemigo")
    body_entered.connect(_on_body_entered)
    $AnimatedSprite2D.play("caminar")

func _process(delta: float) -> void:
    # --- LA IA: perseguir al jugador ---
    var jugador = get_tree().get_first_node_in_group("jugador")
    if jugador == null:
        return
    var direccion = (jugador.position - position).normalized()
    position += direccion * velocidad * delta      # movemos position a mano → con delta
    $AnimatedSprite2D.flip_h = direccion.x < 0

func recibir_dano(cantidad: int) -> void:
    vida -= cantidad
    if vida <= 0:
        morir()

func morir() -> void:
    var jugador = get_tree().get_first_node_in_group("jugador")
    if jugador != null:
        jugador.sumar_kill()
    queue_free()

func _on_body_entered(body: Node) -> void:
    if body.is_in_group("jugador"):
        body.recibir_dano(dano)    # le pega...
        queue_free()              # ...y se sacrifica

```

🗨️ **Cómo “piensa” el slime.** `get_first_node_in_group("jugador")` le devuelve al jugador **sin conocerlo de antemano** (por eso el jugador se anotó en ese grupo). `jugador.position - position` es la **flecha** que va del slime al jugador; `.normalized()` la deja de largo 1, y `* velocidad * delta` la convierte en el pasito de este frame. **Eso es toda la IA.** Se llama *perseguir* (*chase*) y es la base de casi todo enemigo que existe. En la próxima clase, al **élite** le vamos a dar **varios comportamientos** (acechar, perseguir, atacar, sentir el golpe) con una **máquina de estados**.

🗨️ `recibir_dano()` y `morir()` están separados a propósito: la bala va a llamar a la primera, y la segunda avisa al jugador antes de desaparecer. Mañana el **élite** las va a **heredar** tal cual.

## 2.3 · Probarlo

5. Volvé a `nivel.tscn`, seleccioná `Enemigos` y **instanciá** un `enemigo.tscn` adentro (ícono de **cadena** → *Instantiate Child Scene*). Ubicalo en una esquina.
  6. **F6**: el slime **camina hacia vos**. Cuando te toca, tu barra baja 10 y él desaparece.
  7. Cuando funcione, **borrá esa instancia** del árbol: a partir de ahora los va a crear el Spawner.
- ✅ **Punto de control 2:** el slime te persigue, te pega, y la barra de vida del HUD baja.

### 🛑 El slime no se mueve / no me hace daño

#### ▼ Abrí para ver soluciones

- No se mueve → el jugador tiene que estar en el grupo `"jugador"` (lo hace su `_ready`). Si el slime no lo encuentra, `jugador == null` y se queda quieto.
- No hace daño → el `Enemigo` necesita su `CollisionShape2D` **con forma**; sin forma, `body_entered` nunca se dispara. Y el jugador, la suya.
- La animación no arranca → la animación tiene que llamarse `caminar` (es lo que pide el `play()`).

## 🌐 Parte 3 — El Spawner: hordas por los bordes

🗨️ **Recordatorio: ¿qué es *spawnear*?** Viene de *spawn* ("aparecer"). Es **crear una instancia de una escena en tiempo real**, mientras el juego corre — lo hicimos en el TP5 con las piezas que caían. La receta es siempre la misma: `preload()` el molde una vez, y cada tanto `instantiate()` + `add_child()`. Lo nuevo hoy es **dónde**: en vez de arriba, en **cualquiera de los cuatro bordes**.

1. Hijo de `Nivel` → `Node2D` → renombralo `Spawner`.
2. Hijo de `Spawner` → `Timer`. En el Inspector: **Wait Time** `1.0`, **Autostart** ON.

🗨️ En el TP5 creamos el `Timer` **por código** (`Timer.new()`); ahora lo hacemos **como nodo** y lo configuramos en el Inspector. Es exactamente lo mismo — así conocés las dos formas.

3. Clic derecho en `Spawner` → **Attach Script** → `res://spawner.gd`:

```

extends Node2D

var escena_enemigo := preload("res://enemigo.tscn")

func _ready() -> void:
    $Timer.timeout.connect(spawnear)    # cada segundo, un slime

func spawnear() -> void:
    var enemigo = escena_enemigo.instantiate()
    enemigo.position = posicion_en_el_borde()
    get_node("../Enemigos").add_child(enemigo)

func posicion_en_el_borde() -> Vector2:
    var tam := get_viewport_rect().size
    var lado := randi_range(0, 3)        # 0 arriba · 1 abajo · 2 izquierda · 3 derecha
    if lado == 0:
        return Vector2(randf_range(0, tam.x), -40)
    if lado == 1:
        return Vector2(randf_range(0, tam.x), tam.y + 40)
    if lado == 2:
        return Vector2(-40, randf_range(0, tam.y))
    return Vector2(tam.x + 40, randf_range(0, tam.y))

```

💡 `posicion_en_el_borde()` elige un **lado al azar** ( `randi_range` da un entero, `randf_range` un decimal) y devuelve un punto **justo afuera** de la pantalla ( `-40` / `+40` ), así el slime **entra caminando** en vez de aparecer de golpe.

**F6:** cada segundo entra un slime por algún borde y viene por vos. Todavía no podés defenderte: **corré**.

✅ **Punto de control 3:** los slimes aparecen solos por los cuatro bordes, uno por segundo, y te persiguen.

### 🚫 No aparece ninguno / aparecen todos en el mismo lugar

#### ▼ Abrí para ver soluciones

- Ninguno → ¿el `Timer` tiene **Autostart** tildado? ¿Y su `timeout` está conectado a `spawnear` en el `_ready()` ?
- `Cannot load scene res://enemigo.tscn` → revisá que la escena se llame exactamente así y esté en la raíz del proyecto.
- Todos en el mismo lugar → asegurate de que `posicion_en_el_borde()` use `randf_range` y no un número fijo.



## Parte 4 — Balas que apuntan solas

**Concepto:** la bala es otro `Area2D` que vuela recto. El jugador, con un `Timer`, dispara cada 0.4 s **hacia el enemigo más cercano**. No apretás nada: en un *survivors* el arma es automática y vos solo te movés.

### 4.1 · La escena de la bala

1. **Escena** → **Otro Nodo** → `Area2D` → renombralo `Bala`. Guardá como `bala.tscn`.
2. Hijo → `Sprite2D` → **Texture** = `bala.png`, **Scale** `2, 2`.
3. Hijo → `CollisionShape2D` → `CircleShape2D`, radio ≈ `8`.
4. Hijo → `Timer` → renombralo `TimerVida`. **Wait Time** `2`, **One Shot** ON, **Autostart** ON. Es el "tiempo de vida" de la bala: si no le pega a nada, desaparece sola a los 2 s.
5. Clic derecho en `Bala` → **Attach Script** → `res://bala.gd`:

```
extends Area2D

var velocidad := 400.0
var direccion := Vector2.RIGHT # el jugador la cambia al disparar
var dano := 1

func _ready() -> void:
    area_entered.connect(_on_area_entered)
    $TimerVida.timeout.connect(queue_free) # a los 2 s se borra sola

func _process(delta: float) -> void:
    position += direccion * velocidad * delta

func _on_area_entered(area: Area2D) -> void:
    if area.is_in_group("enemigo"):
        area.recibir_dano(dano)
        queue_free()
```

🗨️ `area_entered` es la hermana de `body_entered` (las dos estaban en la tabla de la Clase 4): se emite cuando **otra** `Area2D` entra. Como el slime es un `Area2D`, la bala lo detecta con esta. Si dos balas se cruzan también se emite, pero como no están en el grupo `"enemigo"`, no pasa nada.

### 4.2 · Disparar desde el jugador

6. Hijo de `Jugador` → `Timer` → renombralo `TimerDisparo`. **Wait Time** `0.4`, **Autostart** ON.

7. Reemplazá `jugador.gd` por esta versión (es el mismo de la Parte 1 **más** el disparo, marcado con `#` NUEVO ):

```

extends CharacterBody2D

var velocidad := 200.0
var vida := 100
var kills := 0
var escena_bala := preload("res://bala.tscn")          # NUEVO

func _ready() -> void:
    add_to_group("jugador")
    $TimerDisparo.timeout.connect(disparar)             # NUEVO
    actualizar_hud()

func _physics_process(delta: float) -> void:
    var direccion := Vector2.ZERO
    if Input.is_action_pressed("mover_derecha"):
        direccion.x += 1
    if Input.is_action_pressed("mover_izquierda"):
        direccion.x -= 1
    if Input.is_action_pressed("mover_abajo"):
        direccion.y += 1
    if Input.is_action_pressed("mover_arriba"):
        direccion.y -= 1

    velocity = direccion.normalized() * velocidad
    move_and_slide()

    if direccion != Vector2.ZERO:
        $AnimatedSprite2D.play("run")
        $AnimatedSprite2D.flip_h = direccion.x < 0
    else:
        $AnimatedSprite2D.play("idle")

    var limites := get_viewport_rect().size
    position.x = clamp(position.x, 0, limites.x)
    position.y = clamp(position.y, 0, limites.y)

# ----- NUEVO: el arma automática -----
func disparar() -> void:
    var objetivo = enemigo_mas_cercano()
    if objetivo == null:
        return                                           # no hay a quién dispararle
    var bala = escena_bala.instantiate()
    bala.position = position
    bala.direccion = (objetivo.position - position).normalized()

```

```

    get_node("../Balas").add_child(bala)

func enemigo_mas_cercano():
    var mas_cercano = null
    var menor_distancia := INF
    for enemigo in get_tree().get_nodes_in_group("enemigo"):
        var distancia = position.distance_to(enemigo.position)
        if distancia < menor_distancia:
            menor_distancia = distancia
            mas_cercano = enemigo
    return mas_cercano

# -----

func recibir_dano(cantidad: int) -> void:
    vida -= cantidad
    actualizar_hud()
    if vida <= 0:
        get_tree().reload_current_scene()

func sumar_kill() -> void:
    kills += 1
    actualizar_hud()

func actualizar_hud() -> void:
    get_node("../HUD/LabelKills").text = "Slimes: " + str(kills)
    get_node("../HUD/BarraVida").value = vida

```

🧠 **Cómo elige el objetivo.** `enemigo_mas_cercano()` es un `for` (Clase 2) sobre todos los nodos del grupo `"enemigo"`, guardando el que tenga la **menor distancia** ( `position.distance_to()` ). Arranca con `INF` ("infinito") para que el primer slime siempre gane la comparación. Es el mismo truco de "buscar el mínimo" que se usa en cualquier lenguaje.

**La dirección de la bala** se calcula igual que la del slime, pero al revés: del jugador **hacia** el objetivo.

**F6:** cada 0.4 s sale una bala hacia el slime más cercano. Al pegarle, el slime desaparece y `Slimes:` sube.

✅ **Punto de control 4:** el juego **ya se juega**: esquivás, disparás solo, matás slimes y el contador sube. Si te rodean, tu barra baja y al llegar a 0 se reinicia.

## 🚫 Errores comunes en esta parte

### ▼ Abrí para ver soluciones

- **Las balas no salen** → el `TimerDisparo` tiene que tener **Autostart** y estar conectado en `_ready()`. Y tiene que haber **al menos un slime** vivo: si no, `objetivo == null` y no dispara (es a propósito).
- **Las balas atraviesan a los slimes** → la `Bala` necesita `CollisionShape2D` con forma, y el slime también. Y la bala usa `area_entered`, no `body_entered` (el slime es un `Area2D`).
- **Invalid call. Nonexistent function 'recibir\_dano'** → la bala le pegó a algo del grupo `"enemigo"` que no tiene esa función. Revisá que `enemigo.gd` la tenga.
- **Las balas nunca desaparecen** → `TimerVida` con **One Shot** y **Autostart** tildados, y conectado a `queue_free` en `_ready()`.

## ♥ Parte 5 — El slime élite: herencia + barra de vida

**Concepto:** un enemigo **más duro** que aparece cada tanto. **No lo escribimos de cero: hereda** de `enemigo.gd` (igual que la comida heredaba de `objeto_cae.gd` en el TP5) y solo cambia los números y agrega la **barra de vida sobre la cabeza**.

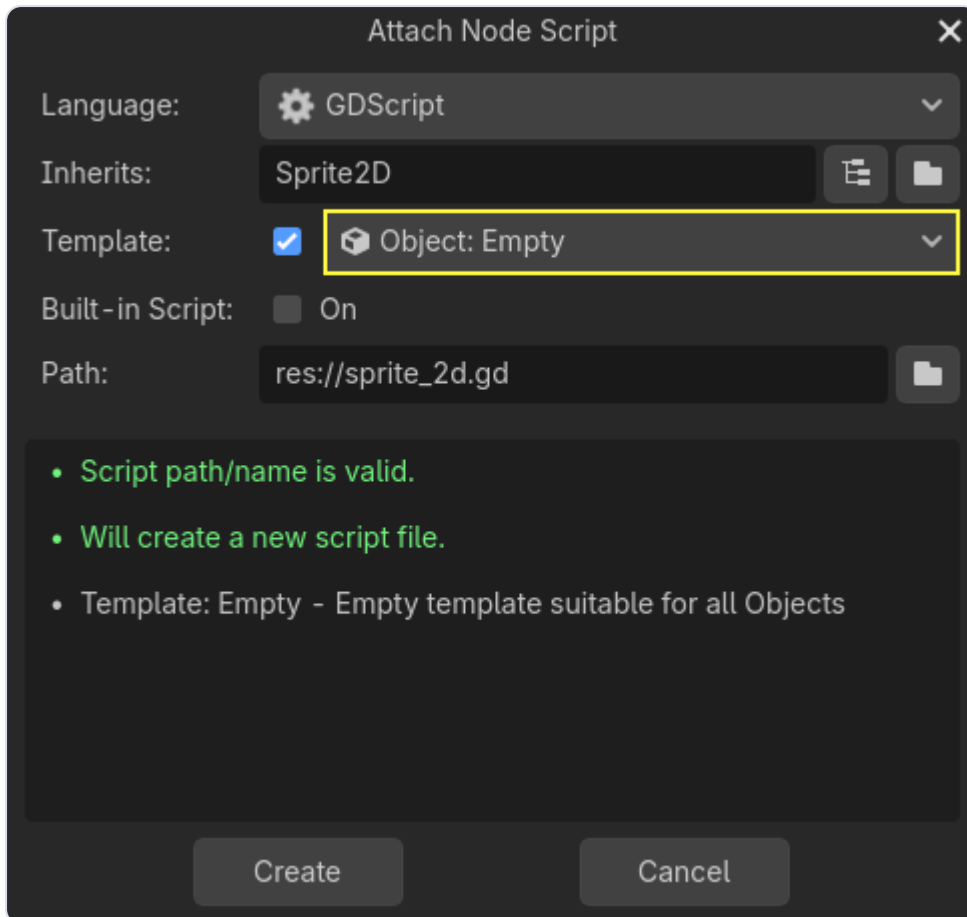
### 5.1 · La escena del élite

1. **Escena** → **Otro Nodo** → `Area2D` → renombralo `EnemigoElite`. Guardá como `enemigo_elite.tscn`.
2. Hijo → `AnimatedSprite2D` → **Nuevo SpriteFrames** → animación `caminar` con `slime_purple.png` (4 × 3, la fila del medio), **Loop**, FPS 8. **Scale** `2.5, 2.5` (es más grande).
3. Hijo → `CollisionShape2D` → `CircleShape2D`, radio ≈ `26`.
4. Hijo → `ProgressBar` → renombralo `BarraVida`. En el Inspector: **Show Percentage** apagado; **Size** ≈ `48 × 6`; **Position** ≈ `-24, -40` (así queda centrada **arriba de la cabeza**).

💡 Un `ProgressBar` es un nodo de UI, pero **puede ser hijo de un nodo 2D**: se dibuja en la posición del padre y **se mueve con él**. Por eso la barra acompaña al slime. Es una barra de vida **en el mundo** (diegética, como la de *Dead Space* de la Clase 1), no en el HUD.

### 5.2 · El script: heredar y ajustar

5. Clic derecho en `EnemigoElite` → **Attach Script**. En el diálogo, en **Inherits** hacé clic en la carpeta 📁 y elegí `enemigo.gd`. Path: `res://enemigo_elite.gd` → **Create**.



## 6. Completá:

```
extends "res://enemigo.gd"

func _ready() -> void:
    super()                # todo lo del slime básico: grupo, señal, animación
    vida = 5                # aguanta 5 balas
    velocidad = 35.0        # más lento
    dano = 25               # pega más fuerte
    $BarraVida.max_value = vida
    $BarraVida.value = vida

func recibir_dano(cantidad: int) -> void:
    super(cantidad)        # resta vida y muere si llega a 0 (lo hace la base)
    $BarraVida.value = vida # y además actualizamos la barra
```

🗨 **Herencia en dos líneas.** `super()` en `_ready()` ejecuta el `_ready()` del slime básico (grupo, señal, animación), y **después** pisamos los valores. En `recibir_dano()`, `super(cantidad)` hace el trabajo de siempre y nosotros solo **agregamos** la barra. La herencia **suma**, no reemplaza — lo mismo que en el TP5.

## 5.3 · Que aparezca cada tanto

7. Reemplazá `spawner.gd` por esta versión (un contador: cada 8 slimes, uno es elite):

```
extends Node2D

var escena_enemigo := preload("res://enemigo.tscn")
var escena_elite := preload("res://enemigo_elite.tscn") # NUEVO
var contador := 0 # NUEVO

func _ready() -> void:
    $Timer.timeout.connect(spawnear)

func spawnear() -> void:
    contador += 1
    var enemigo
    if contador >= 8: # NUEVO: el octavo es elite
        contador = 0
        enemigo = escena_elite.instantiate()
    else:
        enemigo = escena_enemigo.instantiate()
    enemigo.position = posicion_en_el_borde()
    get_node("../Enemigos").add_child(enemigo)

func posicion_en_el_borde() -> Vector2:
    var tam := get_viewport_rect().size
    var lado := randi_range(0, 3)
    if lado == 0:
        return Vector2(randf_range(0, tam.x), -40)
    if lado == 1:
        return Vector2(randf_range(0, tam.x), tam.y + 40)
    if lado == 2:
        return Vector2(-40, randf_range(0, tam.y))
    return Vector2(tam.x + 40, randf_range(0, tam.y))
```

**F6** y aguantá 8 segundos: entra un slime **violeta**, más grande y lento, con su barra encima. Cada bala le baja un quinto; a la quinta, cae.

✅ **Punto de control 5 (final):** cada 8 slimes aparece un elite con barra de vida que **baja bala a bala**, aguanta 5, y al morir suma al contador como cualquier otro.

## Errores comunes con el elite

### ▼ Abrió para ver soluciones

- **La barra no baja** → la función se llama `recibir_dano` en las dos clases (misma firma). Si la escribiste distinto en el elite, no está *sobreescribiendo* nada.
- **El elite no se mueve ni pega** → te olvidaste el `super()` al principio de `_ready()`: sin él, no se anota en el grupo ni conecta la señal.
- **La barra aparece lejos del slime** → ajustá **Position** del `ProgressBar` (negativo en X para centrarla, negativo en Y para subirla).
- `Node not found: BarraVida` → el `ProgressBar` tiene que llamarse **exactamente** `BarraVida` y ser hijo directo de `EnemigoElite`.

## Entrega

Entregá **una** de estas opciones:

1. La **carpeta del proyecto** comprimida en `.zip` (sin `.godot/`), **o**
2. Un **video corto** (o GIF) sobreviviendo ~20 segundos: se ven slimes entrando por los bordes, balas automáticas, el contador subiendo, y un **elite** con su barra bajando hasta morir.

**Nombre:** `tp7-sobrevivir-ApellidoNombre.zip`

### ✓ Checklist de autoevaluación

- ☐ El jugador se mueve en 4 direcciones, animado, y **no sale** de la pantalla.
- ☐ El HUD muestra la **barra de vida** y el **contador de slimes**, y se actualizan en vivo.
- ☐ `enemigo.tscn` es un `Area2D` en el grupo `"enemigo"` que **persigue** al jugador y le pega al tocarlo.
- ☐ El `Spawner` saca un slime por segundo desde un **borde al azar**.
- ☐ Las balas salen **solas** cada 0.4 s hacia el **enemigo más cercano**, y desaparecen a los 2 s si no pegan.
- ☐ Una bala mata a un slime básico y el contador sube.
- ☐ `enemigo_elite.gd` **hereda** de `enemigo.gd` con `extends "res://enemigo.gd"` y `super()`.
- ☐ El elite aparece **cada 8 slimes**, aguanta 5 balas y su **barra sobre la cabeza** baja con cada una.
- ☐ Si tu vida llega a 0, la escena se reinicia.

## Extra (opcional)

---

- **Dificultad que sube:** que el `wait_time` del Spawner baje de a poco (por ejemplo, un 5 % por cada elite que aparece). El pánico de los *survivors* viene de ahí.
  - **Game Over de verdad:** en vez de `reload_current_scene()`, guardá los `kills` en un Autoload y andá a una pantalla de Game Over — todo eso ya lo hiciste en el **TP6**.
  - **Tiempo sobrevivido:** un `Label` en el HUD con los segundos vivos. Pista: una variable que suma `delta` en `_process`.
  - **Un tercer enemigo:** uno **rápido y frágil** (vida 1, velocidad 120), heredando de `enemigo.gd`. Tres líneas.
  - **Golpe visible:** al recibir daño, que el slime parpadee. Los sprites traen un frame **rojo** en la fila de abajo; o usá `modulate` con un `Tween` (Clase 7).
  - **Sonido:** un `AudioStreamPlayer` para el disparo y otro para cuando muere un slime.
- 

## Recursos

---

- Grupos y señales: [Using signals](#) · [Groups](#)
- Instanciar escenas: [Creating instances](#)
- Vectores (dirección, `normalized`, `distance_to`): [Vector math](#)
- El nodo `Timer`: [Timer](#)

Sprites de **Brackeys** (analogStudios\_, RottingPixels) — licencia **CC0**. Capturas del editor: documentación oficial de **Godot Engine**, CC BY 4.0.